

Agentic AI CAD Copilots: Technical Q&A

Comprehensive Questions & Answers Reference Guide: CADABRA, Adam AI & AIDraft Architecture

Q1 What is an Agentic AI System, and how does it differ from traditional LLM generation?

- **Traditional LLM Generation:** Operates in a single passive step (input prompt → output generation) without execution feedback loops, tool access, or memory state.
- **Agentic AI System:** Operates as an autonomous decision-making engine that decomposes goals, invokes external APIs/tools, monitors output errors, and self-corrects until the user's objective is fulfilled.

Q2 Why do AI CAD backends use programmatic API code generation instead of direct 3D mesh output?

- **Parametric Integrity:** CAD software requires fully editable, parametric feature trees with exact geometric constraints, dimensions, and sketch relationships. Direct 3D mesh generation (e.g., `.STL` or `.OBJ`) returns uneditable, "dumb" surfaces.
- **Kernel Native Control:** Translating intent into host CAD APIs (such as SolidWorks COM or Fusion 360 Open API) enables native rebuild engine compatibility, parametric modifications, material assignments, and PDM metadata tracking.

Q3 What are the core specialized agents in a multi-agent CAD architecture, and what are their specific responsibilities?

- **Orchestrator / Router Agent:** Parses natural language intent, extracts dimensional constraints, and delegates sub-tasks across domain agents.
- **CAD Code Generator Agent:** Compiles structured API calls (C#, Python, or C++) to manipulate native CAD features.
- **Vision & Geometric Analysis Agent:** Reads workspace topology, selection face/edge IDs, and Boundary Representation (B-Rep) data from the active document.
- **Standards & Compliance Agent:** Fetches enterprise fastener toolboxes, material profiles, and DFM (Design for Manufacturability) guidelines from internal PLM databases.
- **Debug & Recovery Agent:** Intercepts CAD engine rebuild logs when operations break, executing code healing or state rollbacks dynamically.

Q4 How does a CAD copilot handle error diagnosis and rebuild failures autonomously?

1. **Error Interception:** The plugin catches exception logs or feature rebuild errors directly from the CAD kernel.
2. **Context Serialization:** The Debug Agent captures the broken sketch/feature context along with the error stack trace.
3. **Self-Healing Loop:** The agent generates an amended code snippet to correct broken references or constraint conflicts.
4. **Rollback Fallback:** If healing fails within a set retry threshold (typically 3–6 seconds per loop), the system issues an undo command to return the model to a stable state.

Q5 How do memory and context windows work in a CAD Copilot system?

- **Input Context Window (128k–200k tokens):** Stores large JSON geometry trees, assembly hierarchy histories, full Bill of Materials (BOM) tables, and enterprise drafting rulebooks.
- **Output Token Budget (4,096–8,192 tokens):** Provides sufficient output room to generate procedural scripts modifying dozens of CAD operations in a single execution payload.
- **Stateful Sync:** Local desktop extensions serialize active document metadata over secure WebSockets/HTTPS to keep cloud-hosted agents continuously synced with local workspace updates.

Q6 Which AI CAD Copilot should be selected based on specific operational requirements?

- **Choose CADABRA:** For engineering teams working primarily on **SOLIDWORKS** that require end-to-end automation spanning 3D modeling, assembly updates, and native PDM lookup.
- **Choose Adam AI:** For design teams using **Autodesk Fusion or Onshape** who need clean, optimized parametric feature trees generated directly from natural language prompts.
- **Choose AIDraft:** For enterprise manufacturing, defense, or aerospace environments where the primary bottleneck is converting existing 3D models into audit-ready **2D technical drawings with automated GD&T** under air-gapped/on-premises security constraints.